Part 1 - Solution design: how the donation integration should work

> **The current service assumes everything works first time. My design assumes things will fail, and makes sure every donation is either delivered or visibly in someone's hands.**

Principle: save first, process from a queue, retry what is temporary, escalate what is permanent, and reconcile daily. Every approved donation ends up either in the CRM or visibly with a person, never silently in a log. The diagram is in `integration-design-drawio.png`.

1. Data flow and integration points. After the payment provider approves a payment, the website posts the donation to the integration service over HTTPS. The service **stores it first** and replies `202 Accepted`, so the website never waits on the CRM. A queue feeds a worker that validates, transforms and sends it to the CRM (`POST /api/donors`); on `201` the CRM emails the tax receipt. A daily job also reads the payment provider and the CRM for reconciliation.

2. Authentication and token management. OAuth 2.0 client credentials, with the client secret held in a secrets vault, never in code. One shared token is cached and **refreshed 5 minutes before its 60-minute expiry**, so requests are never sent with an expired token. If a `401` still occurs, the service refreshes once and retries the same donation instead of dropping it, which is what lost 12352, 12371 and 12384.

3. Validation and transformation. Checks: required fields present, email well formed, amount greater than 0, payment approved. Mapping: `amount`, or `otherAmount` when `amount` is null, converted to integer cents (fixing "Other amount" donations such as 12347); first and last name joined into `fullName`; campaign uppercased (`appeal2026` becomes `APPEAL2026`); `submittedAt` becomes `donationDate`; `currency` is always `AUD` and `sourceSystem` is always `WEBSITE`. Invalid data goes straight to review instead of wasting a CRM call.

4. Error handling and retries. Each CRM response has one action: `201` delivered; `409` already exists, treated as success; `400` permanent, so no retry and sent to the review queue; `401` refresh and retry once; `429` wait for `Retry-After`, then retry; `500/503` retry with exponential backoff and jitter (1, 2, 4, 8, 16 s), then the dead-letter queue.

5. Duplicate donations. The donation ID is the idempotency key and is sent as `externalReference`. A donation already delivered is skipped (payload 2 resends 12345), and a `409` is recorded as delivered rather than as an error. Supporters never get two receipts and Finance never double-counts.

6. When the CRM is unavailable or overloaded. Donations wait safely in the queue while the worker retries; after five attempts they move to the dead-letter queue with an alert, and are replayed when the CRM recovers. Replays are safe because of idempotency. For campaign spikes, a shared rate limiter sends at about **90 per minute**, leaving 10% headroom under the CRM's 100 per minute for timing drift, retries and other systems using the same API. The receiving side can scale out (serverless or behind a load balancer), but delivery stays capped by the CRM, so the queue absorbs the spike: receipts may arrive a few minutes late, and nothing is lost.

7. Monitoring and alerting. One structured log line per donation (donation ID, outcome, attempt, CRM ID). Alerts on any review or dead-letter item, rising `429` or `5xx` rates, a growing queue and token refresh failures. A **daily reconciliation** compares payment provider approvals with CRM records by donation ID, count and dollar total, and alerts on any mismatch, so Finance knows the totals are right instead of hearing it from supporters.

8. Security and privacy. HTTPS everywhere, and signed requests from the website so donations can't be forged. Secrets are kept in a vault and rotated. Logs use the donation ID and a masked email, never full supporter details. The service stores only the fields the CRM needs, deletes them once delivery is confirmed, and limits review-queue access to Supporter Care.

Future improvements

1. **Campaign readiness:** load-test before big appeals, and talk to the CRM vendor about a higher limit or a bulk endpoint for campaign days.
2. **Stagger the campaign email:** send it in batches over an hour instead of to everyone at 12:00. The spike disappears at the source, and it costs nothing technically.
3. **Adaptive rate limiting:** start at 90 per minute, slow down automatically if `429`s appear, then speed up again when they stop, and tune the number from real monitoring data.
4. **Payment provider webhook as the source of truth:** today the website tells us about the donation. If the browser closes after payment, we might never hear about it. The payment provider's own webhook is more reliable.
5. **Fix problems at the source:** email validation and a duplicate-submit guard (disable the button, plus an idempotency key) on the website form.
6. **Production hardening:** a managed queue (SQS or Azure Service Bus), a circuit breaker, and a simple replay screen for Supporter Care.